KNOWLEDGE BASE ARTICLE · CROWDSTRIKE NG SIEM DATA ONBOARDING

Integrating Cribl Stream Via Quick Connect to CrowdStrike NG SIEM

AUTHOR	APPLIES TO	DATA USED	VERSION
Shivsai Peddinti	CrowdStrike NG SIEM; Cribl Stream	Cribl Datagen (mock logs)	1.0 · 13 Oct 2026

Contents

- Background
- Data path
- Deployment & Configuration Flow
- Prerequisites: custom parser
- Step 1: Falcon data connection & API credentials
- Step 2: Cribl Quick Connect canvas
- Step 3: Commit & Deploy
- Step 4: Verify ingestion
- Using a real log source
- Troubleshooting
- Quick reference

Background

What is Cribl Stream?

Cribl Stream is an enterprise observability pipeline router that sits between data sources (firewalls, servers, cloud services) and log analytics repositories (like CrowdStrike Falcon Next-Gen SIEM). It gives Security Operations (SecOps) teams complete control over their log data—allowing them to parse, enrich, filter out noise, redact sensitive information, and route telemetry to multiple destinations in real time.

What is the Quick Connect Method?

Quick Connect is Cribl Stream's graphical drag-and-drop routing interface. Instead of writing complex routing logic and regex filter rules manually, Quick Connect allows engineers to visually wire Sources directly to Pipelines and Destinations.

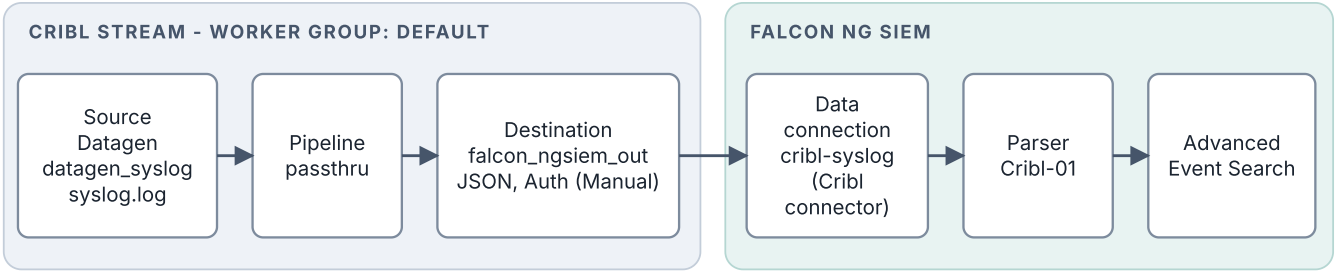
What is Datagen?

Datagen (Data Generator) is Cribl's built-in, license-free log simulator. It generates continuous streams of real-world mock logs (such as RFC 3164 Syslog, Apache Web Access, Palo Alto Firewalls, or AWS VPC Flow logs). It is ideal for staging, testing pipelines, and validating SIEM parser configurations without deploying live agents or vendor licenses.

When is Cribl Stream Used?

A. SIEM Storage & Cost Optimization: Dropping low-value debug/verbose log noise before it reaches CrowdStrike, reducing log storage licenses.
B. Data Sanitization & Masking: Redacting Personally Identifiable Information (PII), credit card numbers, or passwords from logs before cloud entry.
C. Multi-Destination Routing: Forking a single inbound log stream to deliver high-priority alerts to Falcon SIEM while simultaneously archiving raw logs to cheap S3 cloud storage.

Data path



Quick Connect wires the Source to the Destination through the passthru pipeline. The Destination posts JSON to the API URL using the API Key. Datagen is a placeholder for a source that is not yet available. For a real log source, replace the Source tile; the rest is unchanged.

Diagram 1. The data path built in this article.

Deployment & Configuration Flow

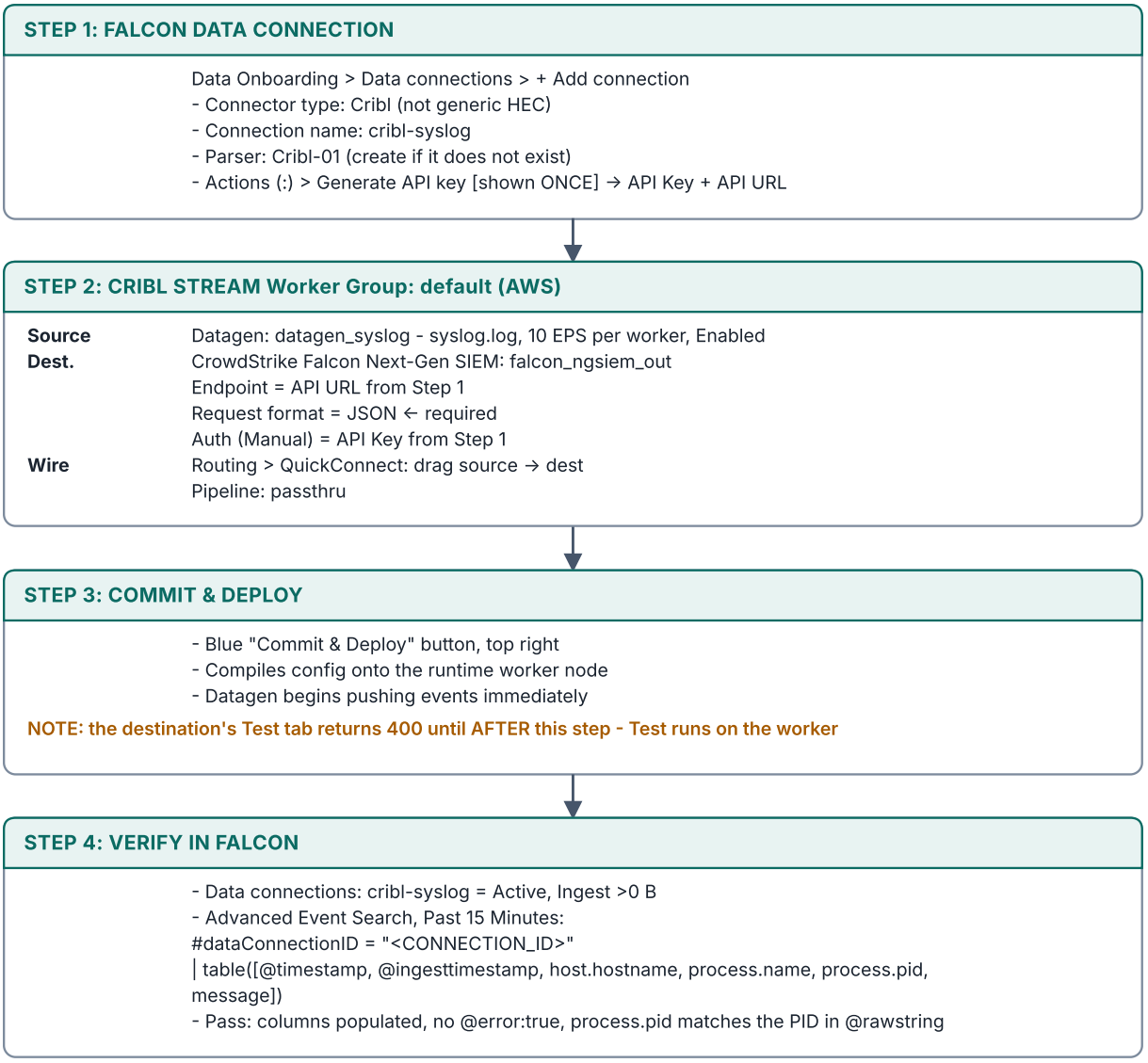


Diagram 2. The four steps at a glance. Each is detailed in Steps 1 to 4 below.

Prerequisites: custom parser

Prerequisites: Make sure a Custom Parser should be created beforehand for the data source:

The parser below is the test parser used in this runbook. Its logic is shown first, then the full source.

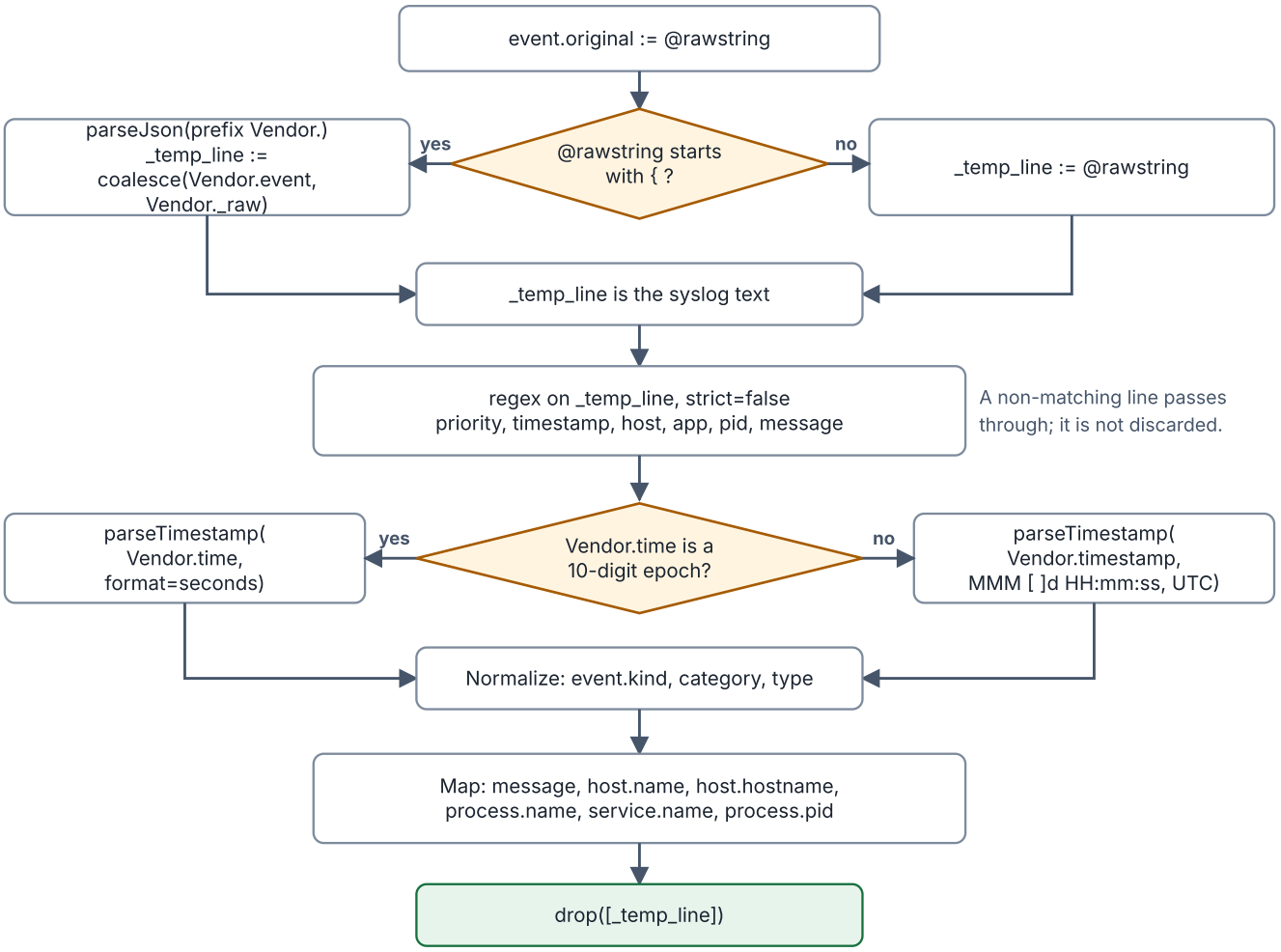


Diagram 3. Parser logic (version 1.0.0). Cribl's destination wraps each event, so the parser handles both a wrapped and a bare syslog line.

Static metadata set by the parser	
Vendor	Cribl (the source marks this VERIFY against developer.crowdstrike.com/docs/ng-siem/vendors)
Parser.version	1.0.0
ecs.version	8.17.0
Cps.version	2.0.0
event.module	syslog
observer.type	forwarder

Source field	Mapped to	Note (from the parser comments)
Vendor.message	`message`	
Vendor.host	`host.name`, `host.hostname`	lower-cased
Vendor.app	`process.name`, `service.name`	
Vendor.pid	`process.pid`	process.pid is integer-typed — parse, don't assign
Vendor.time or Vendor.timestamp	`@timestamp`	envelope epoch if wrapped; otherwise `MMM [ ]d HH:mm:ss` in UTC

Parser source

```
/*
# Cribl Datagen Syslog - TEST parser (mock ingest pipeline validation)
Minimal parser: proves data flows and fields extract. Not production-hardened.
Source: Cribl Datagen `syslog.log` -> Falcon cribl connector
*/

// =========================================================================
// STATIC METADATA
// =========================================================================
| Vendor := "Cribl"                // VERIFY against https://developer.crowdstrike.com/docs/ng-siem/vendors
| Parser.version := "1.0.0"
| ecs.version := "8.17.0"
| Cps.version := "2.0.0"
| event.module := "syslog"
| observer.type := "forwarder"

// =========================================================================
// PREPARSE
// =========================================================================
| event.original := @rawstring

// Cribl's CrowdStrike Falcon Next-Gen SIEM destination wraps each event as {"time":..,"host":..,"event":"<raw syslog>"}.
// The Falcon connector may or may not unwrap it first. Handle both: if it arrives wrapped
// and you only run the syslog regex, nothing extracts while ingestion still looks healthy.
| case {
    @rawstring = /^\s*\{/
        | parseJson(prefix="Vendor.", excludeEmpty="true", handleNull="discard")
        | _temp_line := coalesce([Vendor.event, Vendor._raw]);
    *
        | _temp_line := @rawstring;
}

// RFC3164: <13>Apr 23 14:30:45 hostname app[1234]: message
// strict="false" so a non-matching line is passed through instead of silently discarded —
// the bare `| @rawstring = /.../` form in the template is a filter and drops the record.
| regex(regex="^(?:\\<(?<Vendor.priority>\\d+)\\>)?(?<Vendor.timestamp>[A-Za-z]{3}\\s+\\d{1,2}\\s+\\d{2}:\\d{2}:\\d{2})\\s+
(?<Vendor.host>\\S+)\\s+(?<Vendor.app>[^\\s\\[:]+)(?:\\[(?<Vendor.pid>[^\\]]+)\\])?:?\\s*(?<Vendor.message>.*)",
    field="_temp_line", strict="false")

// =========================================================================
// TIMESTAMP
// =========================================================================
// The template's `Vendor.timestamp := Vendor.REQUIRED_INPUT` line is DELETED — the regex
// above already populates Vendor.timestamp via its named capture.
| case {
    // HEC envelope epoch (seconds, often fractional). Only present if wrapped. No timezone
    // argument: epoch is absolute.
    Vendor.time = /^\d{10}(\.\d+)?$/
        | parseTimestamp(field="Vendor.time", format="seconds");

    // `MMM [ ]d` not `MMM d`: RFC3164 space-pads single-digit days ("Sep  9"), and a plain
    // `d` silently fails to parse every 1st-9th of the month.
    Vendor.timestamp = *
        | parseTimestamp(field="Vendor.timestamp", format="MMM [ ]d HH:mm:ss", timezone="UTC");

    * | *;
}

// =========================================================================
// NORMALIZATION
// =========================================================================
```

```
| event.kind := "event"
| array:append(array="event.category[]", values=["host"])
| array:append(array="event.type[]", values=["info"])

// =============================================================================
// FIELD MAPPING
// =============================================================================
| case { Vendor.message = * | message := Vendor.message; * | *; }
| case { Vendor.host = * | host.name := lower(Vendor.host) | host.hostname := lower(Vendor.host); * | *; }
| case { Vendor.app  = * | process.name := Vendor.app | service.name := Vendor.app; * | *; }
// process.pid is integer-typed — parse, don't assign.
| case { Vendor.pid = /^\d+$/ | parseInt(field="Vendor.pid", as="process.pid", radix=10); * | *; }

// =============================================================================
// CLEANUP
// =============================================================================
| drop([_temp_line])

/*
VALIDATE (SOP Step 4) — Advanced Event Search, Past 15 Minutes:

    #dataConnectionID = "<YOUR_DATA_CONNECTION_ID>"
    | table([@timestamp, @ingesttimestamp, host.hostname, process.name, process.pid, message])

Pass = events appear, those columns are populated, no @error:true.

If host.hostname / process.name are EMPTY but events arrive, the envelope branch is the
thing to check — inspect @rawstring in one event to see whether it starts with `{`.
*/
```

STEP 1: PROVISION FALCON DATA CONNECTION & API CREDENTIALS

1. Log in to the **Falcon Console** and navigate to **NG SIEM > Log Management > Data Onboarding > Data Connection**
2. Click + **Add connection**.
3. Locate the **Cribl Connector** and click **Configure**.
4. Configure the connection parameters:
    a. **Connector name:** cribl-syslog
    b. **Description:** Ingesting mock Syslog events from Cribl Stream via Quick Connect.
    c. **Parsers:** Select your targeted Syslog parser or Click on **Create Parser** to Create one use the above parser example.
5. Accept the **Terms and Conditions** and click **Save**.
6. On the **Data connections** table view, locate cribl-syslog.
7. Click the **Open menu** (:) in the Actions column and select Generate API key.
8. Copy and securely record:
    a. **API Key** (Authorization Bearer Token)
    b. **API URL** (https://ingest.us-2.crowdstrike.com/api/ingest/hec/<CONNECTION_ID>/v1/services/collector)

> **THE API KEY IS SHOWN ONCE**
> Record the API Key when it is generated and keep it in a secure store. Do not paste the key or the full API URL into tickets, chat or screenshots.

STEP 2: CONFIGURE CRIBL QUICK CONNECT CANVAS

1. Log into your **Cribl Stream Workspace** (cribl.cloud or local deployment).
2. Select the **Workspace** suitable for you for this instance we adopted **Testing Environment**
3. Below the Stream and Edge Actions Plane Select **Create source > Worker Group > Default**
4. Select **Datagen** from the available source cards.
5. In the setup menu:
    a. **Input ID**: datagen_syslog
    b. **Data Generator File**: Select syslog.log
    c. Select **Events Per Second Per Worker Node** as 10
    d. **Enable**
6. Click **Save**.

B. Configure the Outbound Destination (Falcon NG- SIEM)

1. Click on Data **(Destinations)**, + Add **Destination**.
2. Select **CrowdStrike Falcon NG SIEM**.
3. Configure the destination settings:
    a. **Output ID:** falcon_ngsiem_out
    b. **URL / Endpoint**: Paste the **API URL** obtained in Step 1.
    c. **Request format**: JSON
    d. **Auth Token**: Paste the **API Key** obtained in Step 1.
4. Click **Save**.

C. Wire Source to Destination via the 'passthru' Pipeline

1. Go to **Routing > Quick Connect canvas**, locate your **datagen_syslog** source card on the left and your **falcon_ngsiem_out** card on the right.
2. Click and drag the connector dot from **datagen_syslog** across the screen and drop it onto **falcon_ngsiem_out**.
3. A connection dialog modal will appear asking you to attach a pipeline:
    a. **Pipeline:** Select **passthru** from the dropdown list.
4. Click **Save**.

STEP 3: COMMIT & DEPLOY CONFIGURATION

1. **Review the canvas:** a line now visually connects **datagen_syslog** through the passthru pipeline directly into **falcon_ngsiem_out**.
2. Look at the top right header bar of the **Cribl interface**: locate the Blue Commit & Deploy button (indicating uncommitted changes).
3. Click **Commit & Deploy**.
4. Enter a descriptive commit message: **Added Quick Connect route**: Datagen Syslog to Falcon NG-SIEM via passthru pipeline.
5. Click **Commit and Deploy**. Cribl Stream will compile the runtime worker node configuration and immediately begin pushing real-time events.

NOTE

The destination's Test tab returns 400 until AFTER this step — Test runs on the worker.

STEP 4: VERIFY INGESTION IN FALCON ADVANCED EVENT SEARCH

1. Return to the Falcon Console and navigate to **Log Management > Data Onboarding > Data connections.**
2. Verify that cribl-syslog displays a green **Active** status.
3. Navigate to **Next-Gen SIEM > Log management > Advanced Event Search.**
4. Set the time picker control to **Past 15 Minutes or 24 Hours.**
5. Execute the following validation query written in **CrowdStrike Query Language (CQL):**

Code snippet

```
#dataConnectionID = "CONNECTION ID"
| table([@timestamp, @ingesttimestamp, host.hostname, process.name, process.pid, message])
```

✓ VALIDATION SUCCESS CHECKLIST

1. Real-time events generated by **Cribl Datagen** appear in the search grid.
2. Fields such as **host.hostname, process.name, and message** are extracted into distinct columns.
3. No **@error:true flags** are present on incoming records.

Using a real log source instead of Datagen

Note: Using a Real Log Source Instead of Datagen

Datagen served as a placeholder for a source that is not yet available. The transport path remains the same for real log sources—including firewalls, servers, appliances, and cloud services—so most of this runbook applies without changes.

The following steps remain unchanged:

- Step 1: Falcon connection and API key
- Step 2B: Destination configuration
- Step 2C: QuickConnect wiring
- Step 3: Commit & Deploy
- Step 4: Verification

Update the following components:

1. **Source tile:** Replace Datagen with the Cribl Source that matches your log source's delivery method: Syslog for firewalls, switches, and Linux; HTTP or Splunk HEC for webhooks; S3 or Azure Blob for object storage; Kafka or Kinesis for message brokers; REST Collector for SaaS APIs; or Windows Event Forwarder for forwarded Windows events.
2. **Parser:** The parser above supports only RFC3164 BSD syslog. Real devices may use RFC5424, CEF, LEEF, JSON, or key=value formats. Before building a custom parser, check whether CrowdStrike already provides one for the vendor. If a custom parser is needed, build it using actual captured samples from Source → Live Data in Cribl or in NG SIEM, rather than relying solely on a format specification.
3. **Network path:** Datagen generates events within Cribl and requires no inbound network connectivity. A real source must be able to reach the Cribl worker. Configure the appropriate listener port, firewall rules, source-IP allow-listing, and TLS where supported.

Troubleshooting

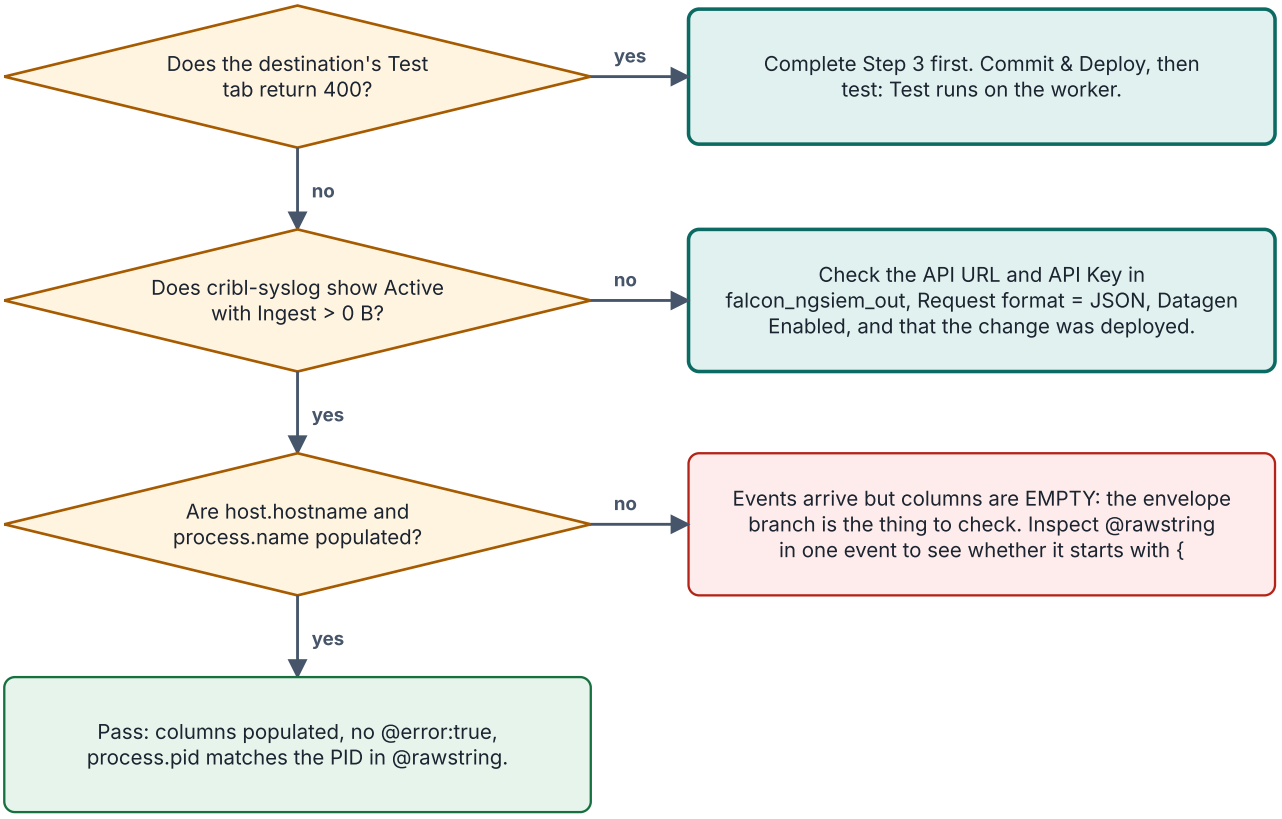


Diagram 4. Start at the top; each answer narrows the cause.

Symptom	Cause	What to do
Destination Test tab returns 400	Test runs on the worker, and the worker has no configuration until Step 3	Complete Step 3 (Commit & Deploy), then Test
Events arrive but `host.hostname` / `process.name` are EMPTY	The envelope branch is the thing to check: the event may still be wrapped in the Cribl envelope	Inspect `@rawstring` in one event to see whether it starts with `{`; the envelope branch of the parser is the thing to check
Nothing extracts but ingestion looks healthy	Cribl's destination wraps each event as {"time":..,"host":..,"event":"<raw syslog>"} and the connector may or may not unwrap it first	Use the parser's envelope handling (handle both shapes)
Events on the 1st to 9th of the month fail to parse	RFC3164 space-pads single-digit days (`Sep  9`); a plain `d` silently fails	Use `MMM [ ]d`, not `MMM d`
Error on `process.pid`	`process.pid` is integer-typed	Parse it with `parseInt`; don't assign
Non-matching lines disappear	The bare `| @rawstring = /.../` form is a filter and drops the record	Use the regex function with `strict="false"` so a non-matching line is passed through

Quick reference

Values used in this runbook	
Connector type	Cribl (not generic HEC)
Connection name	cribl-syslog
Parser	Cribl-01 (create if it does not exist)
Cribl Worker Group	default (AWS); workspace: Testing Environment
Source	Datagen: `datagen_syslog`, `syslog.log`, 10 events per second per worker node, Enabled
Destination	CrowdStrike Falcon Next-Gen SIEM: `falcon_ngsiem_out`
Request format	JSON (required)
Authentication	Manual: API Key from Step 1
Pipeline	passthru
Validation	`#dataConnectionID = "<CONNECTION_ID>"` then `table(...)`, Past 15 Minutes

CrowdStrike and Cribl are trademarks of their respective owners. This article is independent guidance and is not vendor documentation.